

ONBOARDING

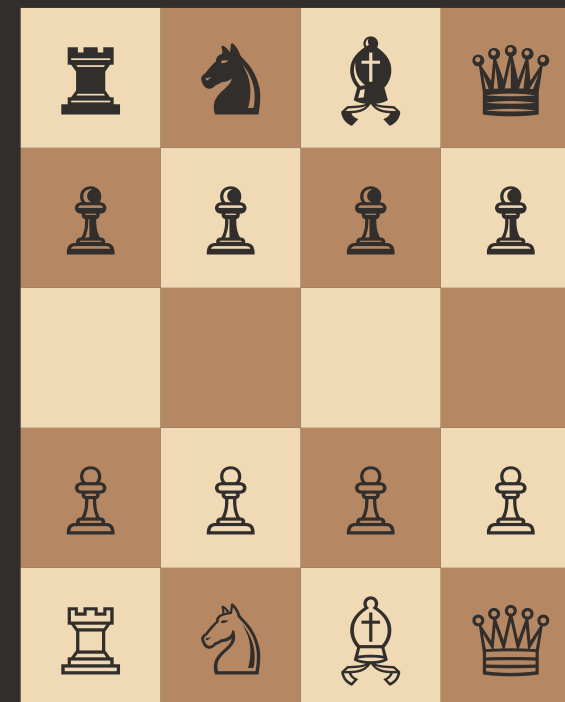
LorChess

From the big picture, to the code, to production.

Part 1 · The big picture

Part 2 · Tech and code

Part 3 · Production



Your homework

- Set up the dev environment: the DEV half of `docs/setup.md` (WSL2, Docker, dev container, Node 24)
- Run `npm install`, the native rebuild, `npm start` and `npm test`. **All green is the first goal.**
- Read `README.md`, skim `docs/design.md`. Full understanding is not expected yet.
- Play the app: a game against LorFish, a PvP game in two browsers, a puzzle, a game review

 marks a point we talk through live. The full written plan is `docs/onboarding.md`.

PART 1

The big picture

What it is, how it fits together, and why it is built this way.

What LorChess is

Play

Against **LorFish**, our engine, or other people in real time. Server clocks, Elo, spectating, chat.

Variants

Standard chess, Chess960, Atomic and Pawn Wars. Odds games with handicaps.

Train

Daily and rated puzzles, game review with Stockfish.

Social

Friends, leaderboard, profiles, achievements.

Membership

Promo codes unlock member features such as game review.

Live

`lorchess.lorand77.dev`

 Who uses it, how many, and what matters to them. Production means real users, real games in progress and real personal data: mistakes are visible.

One request, end to end

```
Browser —DNS (name.com)—> EC2 public IP
      —HTTPS :443—> Caddy (TLS, reverse proxy)
      —HTTP :3000—> Node process (pm2)
                    |— Express: static pages + REST /api/*
                    |— Socket.IO: live games, lobby, chat
                    └— better-sqlite3 —> data/lorchess.sqlite (+WAL)
nightly cron —> backup.sh —> Backblaze B2
```

One server, one process, one database file. No build step, no framework, no bundler.

Where code runs, where state lives

Code runs in three places

- **The server:** Node
- **The browser page:** the UI
- **A browser Web Worker:** LorFish and Stockfish

The chess rules engine runs in all three.

State lives in three places

- **SQLite:** users, sessions, games, moves
- **Server memory:** live rooms, matchmaking, lobby, throttle counters
- **The browser:** its own AI games

Repo map

src/ the server

src/shared/ shared with the browser: rules, engine, catalogues

public/ HTML pages and browser code

test/ unit and integration tests

scripts/ ops scripts (backup)

docs/design.md

why

docs/setup.md

how to run and deploy

docs/CLAUDE.md

rules for working in the repo

docs/licenses.md

third-party licenses

How it is built, and the price

Decision	Buys	Costs
One monolith, one process	Simple to reason about, deploy and debug	In-memory state only works with exactly one process; two would need shared state (e.g. a Socket.IO Redis adapter)
SQLite, not Postgres	No database server to run; backup is one file; synchronous and fast	One writer; the data is tied to one machine's disk. The schema could still move to Postgres.
No framework, bundler or build	What is in git is what runs; nothing to upgrade; small attack surface	More manual DOM code

How it is built, and the price (2)

Decision	Buys	Costs
~7 dependencies, on purpose	Little supply-chain risk and upgrade work	Node built-ins first; sometimes more code of our own
Server decides PvP; browser decides AI games	No cheating in PvP; AI games cost the server nothing	AI-game records are trusted less (e.g. no achievements from a pasted FEN)
Heavy compute in the browser	LorFish and Stockfish cost the server nothing; the server stays small	Depends on the player's device

 If users grow 10×, what breaks first, and what would you change? Which decisions were deliberate from day one, and which were learned the hard way?

How a change ships



What we do not have

- No CI
- No staging environment
- No automated deploy

So the discipline is manual

- Tests run locally, every time
- Deploys are deliberate
- Dev container and server run the same Ubuntu 26.04

PART 2

Tech and code

Node, Express, SQLite, Socket.IO, the shared engine, the front end, security, tests.

One thread, an event loop

- Blocking code freezes **everything**: that is why LorFish runs in a Web Worker
- better-sqlite3 throws synchronously: an unguarded socket handler would take the process and every live game down, so **every handler has a try/catch**
- CommonJS **require** and the module cache: the database path is fixed when **src/db/index.js** first loads
- npm, the lockfile, native modules and why they need a rebuild
- Built-ins replace packages: **--env-file**, **--watch**, **node --test**

The middleware chain: `src/app.js`

1. `trust proxy`: one hop (Caddy)
2. `express.json()`
3. session middleware
4. routers at `/api/<area>`
5. `/js` ← `src/shared/`
6. redirects for old pages
7. static files from `public/`

Trace: `POST /api/login`

throttle → argon2 verify → `session.regenerate` (anti-fixation) → `userId` stored → cookie set; the session is a row in SQLite.

Then `src/server.js` just listens; tests boot `app.js` on a random port.

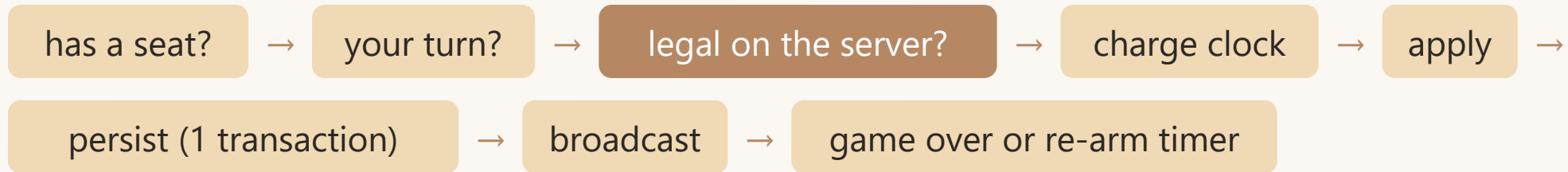
SQLite with better-sqlite3

- WAL mode: the `-wal` and `-shm` files belong to the database
- All SQL is prepared statements in `src/db/queries.js`: no ORM, no SQL injection
- Transactions: a move, the position and both clocks are written together
- **Gotcha:** `schema.sql` only CREATEs. A new column on an existing table needs `addColumnIfMissing`.
- Modelling: the AI side is a real user; deactivation is a flag; deletion renames the account

 Exercise: open the local database with `sqlite3` and query it.

Socket.IO and `handleMove`

Rooms, acks and reconnects. One session cookie serves REST and the socket handshake.



If the write fails, the room rolls back and the mover's ack fails.

Never trust the client. Legality, turn, clocks, time control, variant, handicap and the premove flag are all decided or re-checked on the server.

One file, three runtimes

- `src/shared/chess.js`: the rules
- `src/shared/lorfish.js`: our engine
- Loaded as a browser `<script>`, by a Worker's `importScripts`, and by Node's `require`: the **UMD tail** makes that work
- Catalogues are **allowlists**: time controls, variants, handicaps, achievements
- Positions are rebuilt by **replaying moves**, never by loading a FEN (repetition, Chess960 castling)
- Stockfish: vendored WASM, game review only, **never edited**

Plain JavaScript, one page at a time

- One HTML page, one script in `public/js/`
- `ui.js` never calls the engine or the socket: it talks to a **move source** (AI, remote, spectator)
- Every move goes through one path: `env.applyMove`
- `authGuard.js` is a convenience redirect, **not security**
- DevTools: Network, WebSocket frames, Console

Defence in layers

- **argon2:** slow on purpose
- **Throttle:** per IP, plus a global cap on hashes in flight
- **Cookie:** `httpOnly`, `sameSite`, `secure` via `trust proxy`
- **Username policy:** one generic message, so the lists cannot be probed
- **Privacy page:** its promises are tied to code and config, and change in the same commit

node --test

- **Unit:** pure modules such as the rules, Elo and the catalogues
- **Integration:** the real app on a random port with a throwaway database (`test/helpers/server.js`)
- `npm test` about 20 s · `test:deep` full perft and puzzles · `TEST_LOG=1` shows server logs
- `test:coverage` fails below 98 / 91 / 97 % (lines / branches / functions)



Exercise: write one integration test for an existing endpoint.

How we write code here

- CommonJS, "use strict", double quotes, semicolons
- Every module opens with a comment saying what it does and why
- **No new dependencies** without asking
- Short, imperative commit subjects
- A design change updates `docs/design.md` in the same commit
- AI tools are welcome; **you own what you commit**

PART 3

Production

Where it runs, why there, and how we keep it alive.

What this app needs from a host

One long-running process, long-lived WebSockets, a local disk for SQLite.

Option	Fit	Why
Serverless (Lambda, Vercel)	✗	No long-lived WebSockets, no local disk, no in-memory rooms
PaaS (Railway, Render)	⚠	Works with a persistent volume, but less control. LorChess ran on Railway before.
Containers / Kubernetes	✗	Far too much machinery for one process
VPS / IaaS (EC2, Hetzner)	✓	One process, a local disk, full control

💬 Why we left Railway for EC2 in summer 2026, and why AWS rather than Lightsail or Hetzner.

EC2 t4g.small, Ubuntu 26.04 LTS

Resource	Size	Where it shows
CPU	2 vCPU, ARM (Graviton)	<code>HASH_MAX_CONCURRENT=2</code> . ARM is cheaper per vCPU; native modules must be compiled on the server , never copied from a laptop.
RAM	2 GB	Node, Caddy and in-memory rooms
Disk	8 GB	Database, ~300 MB puzzle import, 7 days of backups, logs, OS updates. The resource to watch.

- **Ubuntu LTS:** long security support, `apt`, the most documentation, same OS as the dev container
- **Node from NodeSource:** Ubuntu's own package lags; NodeSource gives exactly Node 24 LTS

Users and files

Access

- SSH with a key as `ubuntu` (has sudo)
- Work as `lorchess` (no sudo)
- A compromised app gets one user's files, **not the machine**

Filesystem

- `/home/lorchess/lorchess`: the git checkout
- `data/`: database + `-wal`, `-shm`
- `/home/lorchess/backups/`: local backups
- `.env`: `chmod 600`
- `/etc/caddy/Caddyfile`: the proxy

Processes and network

Who runs what

- **systemd** runs Caddy
- **pm2** runs Node: restarts on crash, keeps logs, starts after reboot
- **cron** runs `backup.sh` at 03:00

Network

- DNS A record at name.com → instance IP
- Open: **22, 80, 443**. Port **3000 stays closed**: the throttle trusts `X-Forwarded-For` only because Caddy is the sole way in.
- Caddy gets and renews Let's Encrypt certificates (port 80)

 Is the IP an Elastic IP? If not, a stop and start changes it and DNS breaks.

Config and secrets

- `.env` holds only `SESSION_SECRET` and `NODE_ENV=production`
- Everything else has a default in `src/config.js`
- Rotating `SESSION_SECRET` logs **everyone** out
- `NODE_ENV=production` hides stack traces from responses

Never commit the secret, and never paste it into chat or AI tools.

What a deploy does

git pull → npm install → pm2 restart lorches

- **Every socket drops.** Live PvP games are rebuilt from the database when players return; those not resumed within 10 min are aborted. **Deploy at quiet times.**
- **Schema changes run at boot** through `addColumnIfMissing`: add-only, so rollback is safe
- **Rollback:** `git checkout <previous>`, then `pm2 restart`
- If native module versions change, a rebuild may be needed

Admin commands

- `user:password`
- `user:deactivate` / `user:reactivate`
- `user:delete`: **verify the owner first**
("change your board colours to a value I name")
- `promo:new`

Safe while the app runs. Run as `lorchess` in the project directory, so `.env` is picked up.

Never against production: `db:reset`.

`puzzles:import` downloads ~300 MB: only after asking.

Backups

cron 03:00 → **VACUUM INTO** → local copy (7 days) → rclone → B2 (30 days)

- **VACUUM INTO** gives a consistent copy while the app keeps writing. Why is **cp** of a live WAL database wrong?
- **Data we can lose:** up to about 24 hours
- **Time to recover:** however long a restore takes

Restore, and privacy

pm2 stop → copy backup over the database → delete -wal and -shm → pm2 start

Restore drill

Restore last night's B2 backup into **your dev environment** and check it works. A backup is only known to work once a restore has been tested.

Privacy

Backups hold personal data. The policy promises at most 90 days; deleted users leave backups by expiry. Caddy keeps no access log.

Logs, monitoring, updates

Logs

`pm2 logs`

`pm2 monit`

💬 Log rotation? On 8 GB, logs never rotated will fill the disk.

Monitoring

Disk: `df -h`

The backup log

💬 Who finds out the site is down, and how? An uptime check is cheap.

Updates

Ubuntu security patches, Node versions, npm packages (7-day release age, Socket scanner)

💬 unattended-upgrades?

What happens, and what do we do?

The Node process crashes

A bad deploy goes out

The disk fills up

The certificate fails to renew

`SESSION_SECRET` leaks

Someone hammers the login endpoint

The instance dies completely: rebuild from `docs/setup.md` plus a B2 restore. That is why `setup.md` must stay accurate.

Incident habit: reproduce locally → logs and stack trace → failing test → fix → deploy → a short note.

Growing into production

shadow a deploy → deploy while supervised → deploy alone

The same steps for the admin commands and the restore drill.

💬 Which access, and when: GitHub (push to **main**?), SSH to production, the AWS console, B2, name.com.

PLAN

Three weeks

Week 1

Part 1 (sections 1–4), Node and Express (5–6).

Exercise: a small REST change.

Week 2

Sections 7–13.

A first real ticket.

Week 3

Part 3 (sections 14–19).

Restore drill, shadowed deploy, failure scenarios.

PLAN

Good first tasks

- Add a stats field from the API through to the profile page
- Cover a branch the coverage report shows as untested
- Add an achievement: catalogue, service, toast, backfill
- Ops: write the missing runbook items from section 18 (log rotation, an uptime check)

QUESTIONS?

Welcome aboard

[docs/onboarding.md](#) · [docs/design.md](#) · [docs/setup.md](#) · [docs/CLAUDE.md](#)